

STM – DOCKER E KUBERNETES

Instrutoria Interna

Construção de Imagens e Aplicações com Docker
Camadas, Dockerfile, Volumes e Redes

Wescley de Macêdo Sousa
wescleysousa@stm.jus.br



**STM – SUPERIOR TRIBUNAL
MILITAR**

DITIN/COTEC

Criado em 20/09/2026

CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

SUMÁRIO

MÓDULO 1: ANATOMIA E CONSTRUÇÃO DE IMAGENS DOCKER	3
1.1 – ANATOMIA DE UMA IMAGEM DOCKER E O SISTEMA DE CAMADAS	3
1.2 – O DOCKERFILE E SUAS PRINCIPAIS INSTRUÇÕES	4
1.4 – BOAS PRÁTICAS AVANÇADAS NA CRIAÇÃO DE IMAGENS	9
MÓDULO 2: PERSISTÊNCIA DE DADOS E ARQUITETURA DE REDES DOCKER	12
2.1 PERSISTÊNCIA DE DADOS EM CONTAINERS	12
2.2 TIPOS DE ARMAZENAMENTO: VOLUMES VS BIND MOUNTS VS TMPFS	13
2.3 ARQUITETURA DE REDES DOCKER	15
2.4 COMUNICAÇÃO ENTRE CONTAINERS E RESOLUÇÃO DE NOMES (DNS INTERNO)	16
MÓDULO 3: LABORATÓRIO E PRÁTICAS GUIADAS	17
3.1 ATIVIDADE PRÁTICA 1: CONSTRUÇÃO DE IMAGEM E CICLO DE VIDA COMPLETO	17
3.2 ATIVIDADE PRÁTICA 2 (DESAFIO MULTICONTAINER): APLICAÇÃO WEB + BANCO DE DADOS	23
RESUMO DE COMANDOS DO MÓDULO	30
REFERÊNCIAS	31



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

MÓDULO 1: ANATOMIA E CONSTRUÇÃO DE IMAGENS DOCKER

1.1 – ANATOMIA DE UMA IMAGEM DOCKER E O SISTEMA DE CAMADAS

Uma **Imagem Docker** é um pacote estático, imutável e autocontido que reúne todos os elementos necessários para a execução de um software: código-fonte, binários, bibliotecas do sistema operacional, variáveis de ambiente e arquivos de configuração.

Diferente de um arquivo ISO de máquina virtual tradicional (que representa um único bloco contínuo de disco), a imagem Docker é construída sobre uma arquitetura de **camadas empilhadas (Layered Image Architecture)** suportada por um sistema de arquivos de união, conhecido como **Union File System (UnionFS)** — com destaque para o driver **OverlayFS (Overlay2)** no Linux moderno.

CAMADA GRAVÁVEL DO CONTAINER (Container Writable Layer)	<--- Read-Write (RW)
Instrução CMD ["node", "server.js"] (Metadata)	<--- Read-Only (RO)
Instrução COPY . . (Código da Aplicação)	<--- Read-Only (RO)
Instrução RUN npm install (Bibliotecas / node_modules)	<--- Read-Only (RO)
Instrução WORKDIR /app (Metadata)	<--- Read-Only (RO)
Instrução FROM node:18-alpine (Sistema Operacional Base)	<--- Read-Only (RO)

O Mecanismo do OverlayFS (LowerDir, UpperDir e Merged)

O driver OverlayFS combina diferentes diretórios no host para apresentar uma visão unificada do sistema de arquivos ao container:

1. **LowerDir (Camadas de Imagem - Read-Only):** Representa o conjunto de camadas estáticas da imagem base. Essas camadas nunca são alteradas diretamente.
2. **UpperDir (Camada do Container - Read-Write):** É a camada superior gravável criada no momento em que o container é instanciado. Qualquer arquivo novo ou alterado em tempo de execução é salvo exclusivamente aqui.
3. **MergedDir (Visão Unificada):** É o ponto de montagem apresentado ao processo dentro do container, fundindo a visão das camadas inferiores (**LowerDir**) com a camada superior (**UpperDir**).
4. **WorkDir:** Diretório auxiliar utilizado pelo Kernel para gerenciar transações e operações internas de cópia.

Estratégia Copy-on-Write (CoW)



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

A estratégia **Copy-on-Write** (**Cópia na Escrita**) garante eficiência extrema de memória e espaço em disco:

- Se um processo dentro do container tenta ler um arquivo existente na imagem base, o sistema o lê diretamente da camada **Read-Only** (**LowerDir**).
- Se o processo tenta **modificar** um arquivo vindo da imagem base, o OverlayFS copia primeiramente esse arquivo da camada inferior (**LowerDir**) para a camada superior gravável (**UpperDir**) e aplica a alteração no arquivo copiado.
- O arquivo original na camada inferior permanece intacto, garantindo que a imagem original não seja corrompida e possa ser compartilhada simultaneamente por dezenas de outros containers.

1.2 – O DOCKERFILE E SUAS PRINCIPAIS INSTRUÇÕES

O **Dockerfile** é um arquivo de texto declarativo, sem extensão, que contém a sequência ordenada de instruções para a compilação e criação automatizada de uma imagem Docker.

Tabela de Instruções Fundamentais:



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Instrução	Fase de Execução	Descrição e Propósito
FROM	Build	Define a imagem base inicial da qual esta imagem herdar a estrutura.
WORKDIR	Build	Define o diretório de trabalho padrão dentro da imagem para as instruções seguintes.
COPY	Build	Copia arquivos/diretórios do sistema de arquivos do Host para a imagem.
RUN	Build	Executa comandos no sistema de arquivos da imagem durante a compilação (cria novas camadas).
ENV	Build / Runtime	Define variáveis de ambiente persistentes no container.
EXPOSE	Documentação	Sinaliza qual porta de rede o container escuta por padrão (não publica a porta no host).
CMD	Runtime	Fornecer os argumentos padrão ou o comando executado ao iniciar o container.
ENTRYPOINT	Runtime	Configura o executável principal fixo que rodará dentro do container (<code>PID 1</code>).

Detalhamento das Instruções com Exemplos Práticos:

1. FROM

Toda imagem válida deve começar com a instrução FROM. Ela resgata uma imagem existente do registro local ou do Docker Hub.



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Dockerfile

```
# Define a imagem oficial do Node.js versão 18 baseada no Linux Alpine  
FROM node:18-alpine
```

2. WORKDIR

Cria e estabelece o diretório interno onde os comandos subsequentes (RUN, COPY, CMD) serão executados. Evita o uso excessivo de `cd /caminho && comando`.

Dockerfile

```
# Define o diretório interno da aplicação  
WORKDIR /usr/src/app
```

3. COPY

Sintaxe: `COPY <origem_no_host> <destino_no_container>`. Transfere arquivos locais para dentro da imagem.

Dockerfile

```
# Copia os arquivos de definição de dependências para o diretório de trabalho atual  
COPY package.json package-lock.json ./
```

💡 **Diferença entre COPY e ADD** : Dê preferência absoluta ao COPY . A instrução ADD possui comportamentos adicionais implícitos, como extrair automaticamente arquivos compactados (`.tar.gz`) e aceitar URLs remotas, o que pode gerar vulnerabilidades de segurança e comportamentos inesperados.



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

4. RUN

Executa comandos no shell da imagem em tempo de compilação. Cada instrução **RUN** gera uma nova camada imutável no disco.

Dockerfile

```
# Atualiza os índices do gerenciador de pacotes e instala dependências sem cache  
RUN apk add --no-cache curl && npm install --production
```

5. ENV

Define variáveis de ambiente que estarão disponíveis durante o *build* e no ambiente de *runtime* do container.

Dockerfile

```
ENV NODE_ENV=production  
ENV PORT=3000
```

6. EXPOSE

Funciona primariamente como documentação entre o desenvolvedor e a equipe de infraestrutura. Não abre ou publica a porta no host por si só.

Dockerfile

```
EXPOSE 3000
```

7 e 8. CMD vs ENTRYPOINT (Análise Comparativa)



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Ambas as instruções definem a execução inicial do container, mas comportam-se de maneiras distintas quanto à sobrescrita via CLI:

- **Forma Exec (Exec Form - Recomendada):** Utiliza sintaxe de vetor em JSON: ["executavel", "param1", "param2"]. O processo é iniciado como PID 1 no Linux, recebendo sinais de término (SIGTERM) diretamente.
- **Forma Shell (Shell Form):** Utiliza texto puro: executavel param1. O comando é envolvido por um subshell (/bin/sh -c), o que impede o processo de receber sinais de desligamento gracioso.

Dockerfile

```
# Forma Exec  
CMD ["npm", "start"]
```

Combinação Avançada: ENTRYPOINT + CMD

A melhor prática corporativa combina as duas instruções. O ENTRYPOINT estabelece o comando fixo imutável e o CMD fornece os parâmetros padrão que podem ser sobrescritos pelo usuário na linha de comando.

Dockerfile

```
# ENTRYPOINT define o executável imutável  
ENTRYPOINT ["ping"]  
  
# CMD define o argumento padrão (pode ser substituído ao rodar o container)  
CMD ["localhost"]
```

Se executado com `docker run meu-ping`: O container executa `ping localhost`.



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Se executado com `docker run meu-ping 8.8.8.8`: O container executa `ping 8.8.8.8` (o parâmetro `8.8.8.8` sobrescreve o `CMD`).

Host (CLI) —(Envia Build Context)—> Docker Daemon —(Processa instruções)—> Gera Imagem no Local Storage

O Algoritmo de Cache

O Docker reutiliza camadas existentes em builds anteriores para acelerar a compilação. A decisão do reuso do cache ocorre da seguinte forma:

1. O Docker analisa a instrução atual e compara com todas as imagens filho geradas previamente no host.
2. Se a instrução for `COPY` ou `ADD`, o Docker calcula o checksum SHA-256 de cada arquivo copiado. Se o arquivo no host sofreu alteração de um único byte, o cache daquela camada é **invalidado**.
3. **Efeito Cascata**: Assim que o cache de uma camada é invalidado, **todas as instruções subsequentes no Dockerfile serão reexecutadas obrigatoriamente**, ignorando o cache do restante do arquivo.

Plaintext

```
[ Instalação do SO (FROM) ]      ---> Cache OK
[ Instalação de Pacotes (RUN) ]  ---> Cache OK
[ Cópia do package.json ]       ---> Cache OK (Sem alteração)
[ npm install (RUN) ]           ---> Cache OK
[ Cópia do Código-fonte (COPY) ] ---> ALTERADO! (Invalida Cache)
[ Configuração Final (ENV/CMD) ] ---> Re-executa obrigatoriamente (Sem Cache)
```

1.4 – BOAS PRÁTICAS AVANÇADAS NA CRIAÇÃO DE IMAGENS

1. Ordenação Inteligente de Camadas

Posicione as instruções que mudam com menor frequência no início do Dockerfile e as que mudam com alta frequência (código-fonte) no final.



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

2. Otimização do .dockerignore

Assim como o .gitignore, o arquivo .dockerignore evita que arquivos desnecessários (logs, diretórios de compilação local como node_modules, pastas .git, arquivos sensíveis .env) sejam enviados no *Build Context* para o Daemon.

```
Plaintext

# Arquivo .dockerignore de exemplo
.git
.gitignore
node_modules
npm-debug.log
Dockerfile
.env
```

3. Builds em Múltiplos Estágios (Multi-Stage Builds)

Permite utilizar imagens pesadas completas para compilar a aplicação e, no mesmo Dockerfile, copiar **apenas o artefato gerado** para uma imagem final extremamente enxuta.



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Dockerfile

```
# --- ESTÁGIO 1: Compilação (Build) ---
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build # Gera a pasta dist/

# --- ESTÁGIO 2: Runtime (Produção) ---
FROM nginx:alpine
# Copia apenas os arquivos estáticos gerados do estágio 'builder'
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Resultado: Redução drástica da imagem de ~900 MB para ~25 MB, eliminando compiladores e ferramentas do ambiente final de produção.

4. Execução como Usuário Não-Root

Por padrão, os containers executam como root. Em ambientes corporativos seguros, defina um usuário sem privilégios via instrução USER.

Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY . .
# Altera para o usuário padrão não-privilegiado 'node' da imagem Alpine
USER node
CMD ["node", "app.js"]
```

1.5 Publicação e Utilização de Imagens



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Após compilar a imagem, você pode versioná-la e publicá-la em um *Registry* corporativo ou no Docker Hub.

```
Bash

# 1. Autenticar no Registro
docker login

# 2. Criar uma Tag alinhada ao repositório remoto (Sintaxe: usuario/repositorio:tag)
docker tag minha-aplicacao:latest wescley/minha-aplicacao:v1.0.0

# 3. Enviar a imagem compilada para o Registry
docker push wescley/minha-aplicacao:v1.0.0
```

MÓDULO 2: PERSISTÊNCIA DE DADOS E ARQUITETURA DE REDES DOCKER

2.1 PERSISTÊNCIA DE DADOS EM CONTAINERS

Como aprendido no Módulo 1, a camada gravável superior de um container (Container Writable Layer) possui natureza **efêmera**. Se o container for apagado (`docker rm`), todas as gravações ocorridas durante o seu ciclo de vida são irremediavelmente perdidas.

Para permitir que aplicações com estado (*stateful*) — como PostgreSQL, MySQL ou sistemas de upload de arquivos — mantenham seus dados intactos após a destruição dos containers, o Docker disponibiliza estratégias de armazenamento fora do ciclo de vida do container.



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

2.2 TIPOS DE ARMAZENAMENTO: VOLUMES VS BIND MOUNTS VS TMPFS

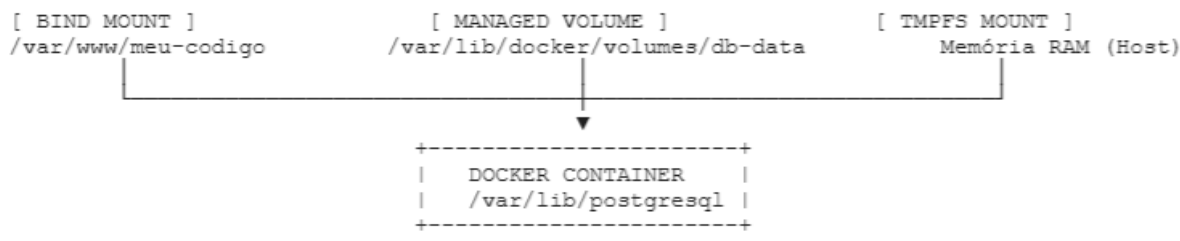


Tabela Comparativa de Estratégias:

Característica	Managed Volumes (<code>docker volume</code>)	Bind Mounts	tmpfs Mount
Local de Armazenamento no Host	Gerenciado pelo Docker (<code>/var/lib/docker/volumes/</code>).	Qualquer caminho do sistema de arquivos do Host.	Apenas na memória RAM do Host.
Portabilidade entre Hosts	Alta (Compatível com drivers de Nuvem/NFS).	Baixa (Depende do caminho exato da máquina).	Nula (Restrito ao host local).
Gerenciamento via CLI	Sim (<code>docker volume create/ls/rm</code>).	Não (Controlado pelo sistema de arquivos do Host).	Não.
Desempenho de I/O	Nativo e Alto.	Nativo e Alto (Pode ser lento no macOS/Windows).	Extremamente Alto (Velocidade de RAM).
Caso de Uso Recomendado	Bancos de dados e dados de produção.	Ambiente de Desenvolvimento em Tempo Real (Live Reload).	Armazenamento temporário de senhas/chaves em RAM.

Utilizando Volumes Gerenciados (`docker volume`)



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Bash

```
# Criar um volume explicitamente
docker volume create dados-postgres

# Listar os volumes gerenciados existentes no host
docker volume ls

# Inspecionar o local físico onde o Docker guarda os dados do volume
docker volume inspect dados-postgres
```

Montando um Volume no Container (Sintaxe --mount vs -v):

A sintaxe moderna --mount é recomendada por ser mais explícita e legível do que a sinalização legada -v.

Bash

```
# Executando um banco PostgreSQL com Volume Gerenciado
docker run -d \
  --name banco-prod \
  -e POSTGRES_PASSWORD=senha_segura \
  --mount source=dados-postgres,target=/var/lib/postgresql/data \
  postgres:15-alpine
```



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Utilizando Bind Mounts para Desenvolvimento em Tempo Real

Bash

```
# Mapeia o diretório atual do projeto no host diretamente para a pasta /app dentro do container
docker run -d \
  --name api-dev \
  -p 3000:3000 \
  --mount type=bind,source="$(pwd)",target=/app \
  node:18-alpine
```

💡 **Vantagem do Bind Mount no Desenvolvimento:** Qualquer alteração efetuada pelo desenvolvedor em seu editor de código na máquina host é refletida instantaneamente dentro do container, acionando ferramentas de Live Reload (como `nodemon`).

2.3 ARQUITETURA DE REDES DOCKER

O Docker abstrai a complexidade do subsistema de redes do Linux (iptables, pontes virtuais, interfaces veth) fornecendo drivers de rede plugáveis.

Plaintext

DRIVERS DE REDE DOCKER

[BRIDGE]

Rede isolada padrão
com NAT para containers

[HOST]

Compartilha a interface
de rede do Host

[NONE]

Isolamento total
Sem placa de rede

Drivers Principais:

1. **bridge (Padrão):** Cria uma interface de rede virtual do tipo ponte (geralmente chamada `docker0`) no host. Cada container conectado a uma rede *bridge* recebe



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

um IP interno privado na sub-rede virtual. O Docker gerencia regras de **NAT (Network Address Translation)** no iptables do host para permitir acesso à internet e expor portas.

2. **host:** Remove o isolamento de rede entre o container e o sistema hospedeiro. O container não recebe um IP próprio; ele utiliza diretamente as interfaces de rede e as portas do host.

Bash

```
# O container Nginx escutará diretamente na porta 80 do IP real da máquina  
docker run -d --net=host nginx
```

1. **none:** Desativa completamente a pilha de rede dentro do container. O container possui apenas a interface de *loopback* (127.0.0.1), sem qualquer conectividade externa ou interna.
2. **overlay:** Utilizado em ambientes distribuídos (Docker Swarm / Kubernetes) para conectar containers rodando em **múltiplos hosts físicos diferentes** na mesma rede privada criptografada.

2.4 COMUNICAÇÃO ENTRE CONTAINERS E RESOLUÇÃO DE NOMES (DNS INTERNO)

⚠ Regra Fundamental de Redes no Docker:

Na rede `bridge` padrão do Docker (`default bridge`), os containers não conseguem se comunicar utilizando seus **NOMES** (*não há resolução de DNS automática*), apenas via *Endereço IP* diretamente.

Para obter Resolução de Nomes via DNS Interno Automático, é **OBRIGATÓRIO** criar uma Rede Customizada (User-Defined Bridge Network).

CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Criando e Gerenciando Redes Customizadas:

```
Bash

# 1. Criar uma rede customizada do tipo bridge
docker network create minhanetwork-app

# 2. Listar as redes ativas no host
docker network ls

# 3. Inspecionar a sub-rede alocada e os containers conectados
docker network inspect minhanetwork-app
```

Conectados a uma rede customizada, o container webapp pode se conectar ao container db simplesmente utilizando o endereço db:5432 como hostname na string de conexão do banco de dados, sem se preocupar com mudanças dinâmicas de IP.

MÓDULO 3: LABORATÓRIO E PRÁTICAS GUIADAS

3.1 ATIVIDADE PRÁTICA 1: CONSTRUÇÃO DE IMAGEM E CICLO DE VIDA COMPLETO

Nesta atividade, você construirá a imagem de uma API funcional desenvolvida em Node.js e explorará os comandos de inspeção e auditoria.



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Passo 1: Preparação da Aplicação Local

Crie uma pasta de trabalho em sua máquina local e adicione os seguintes arquivos:

1. Arquivo `package.json`:

JSON

```
{
  "name": "api-simples",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

1. Arquivo `server.js`:



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

JavaScript

```
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.json({
    status: 'Sucesso',
    mensagem: 'API Rodando em Container Docker!',
    timestamp: new Date()
  });
});

app.listen(PORT, () => {
  console.log(`Servidor rodando com sucesso na porta ${PORT}`);
});
```

1 - Arquivo Dockerfile:

Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Passo 2: Compilar a Imagem (docker build)

No terminal, dentro da pasta do projeto, execute o comando de compilação atribuindo a tag api-simples:v1:



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Bash



```
docker build -t api-simples:v1 .
```

Passo 3: Verificar a Imagem Gerada (docker images)

Bash



```
docker images
```

Validação: Verifique se a imagem `api-simples` com a tag `v1` está listada e observe o seu tamanho em disco.

Passo 4: Instanciar e Executar o Container (docker run)

Inicie o container em modo desconectado (*detached*), nomeando-o como `api-container` e mapeando a porta 8080 do host para a porta 3000 do container:

Bash



```
docker run -d --name api-container -p 8080:3000 api-simples:v1
```

Acesse a aplicação no seu terminal ou navegador:

Bash



```
curl http://localhost:8080
```



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Passo 5: Executar Comandos Dentro do Container Ativo (docker exec)

O comando `docker exec` permite entrar no container em execução ou rodar comandos dentro do seu ambiente isolado sem pará-lo.

Bash



```
# Executa um comando simples de inspeção de arquivos dentro do container
docker exec api-container ls -la /app

# Abre uma sessão interativa no Shell (Alpine utiliza 'sh') dentro do container
docker exec -it api-container sh
```

Dentro do container, teste:

Bash



```
# Verifique o usuário atual
whoami

# Saia da sessão do container
exit
```

Passo 6: Auditar os Logs da Aplicação (docker logs)

Exiba os logs do stdout produzidos pelo Express:

Bash



```
docker logs api-container
```



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Para acompanhar os logs em tempo real enquanto faz requisições em outra janela do terminal, utilize a flag -f:

Bash



```
docker logs -f api-container
```

Passo 7: Inspecionar o JSON de Metadados (docker inspect)

Inspeccione todas as propriedades do container ativo:

Bash



```
docker inspect api-container
```

Extraindo o status exato de execução e o IP via filtro de saída:

Bash



```
docker inspect --format='Status: {{.State.Status}} | IP: {{.NetworkSettings.Networks
```

Bash



```
{{.State.Status}} | IP: {{.NetworkSettings.Networks.bridge.IPAddress}}' api-container
```

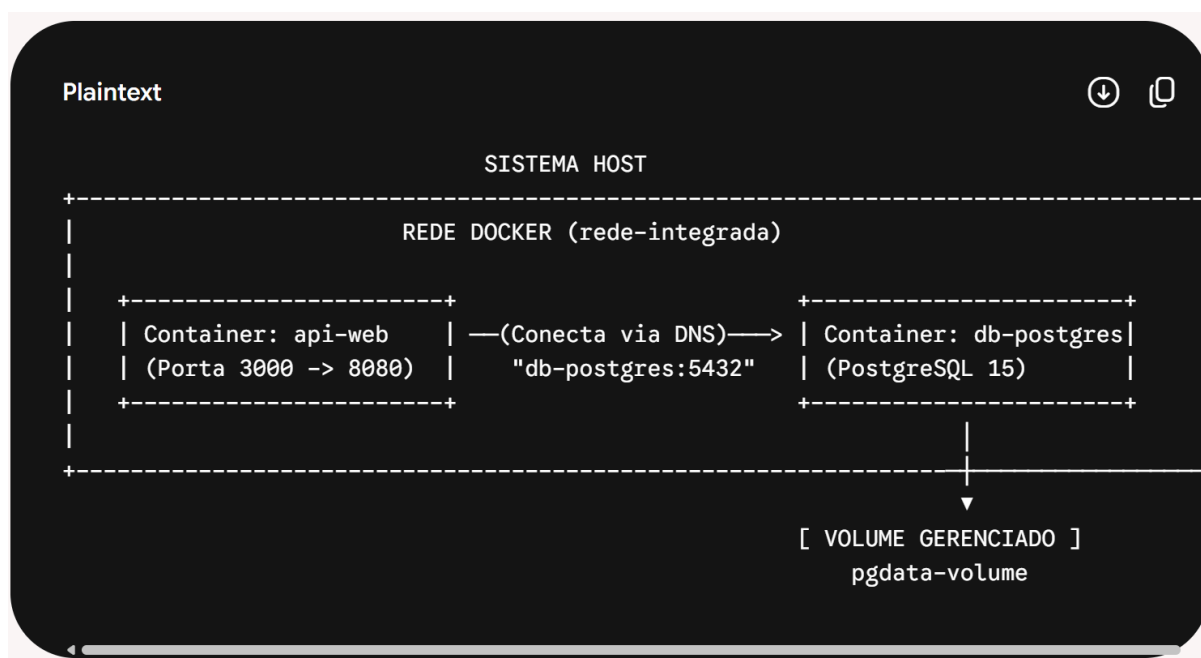


CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

3.2 ATIVIDADE PRÁTICA 2 (DESAFIO MULTICONTAINER): APLICAÇÃO WEB + BANCO DE DADOS

Objetivo do Desafio: Construir um ambiente completo contendo dois serviços interdependentes sem utilizar frameworks de orquestração superior (como Docker Compose):

1. Um **Banco de Dados PostgreSQL** que persiste seus dados em um **Volume Gerenciado**.
2. Uma **API Web** conectada ao PostgreSQL através de uma **Rede Customizada**, utilizando o DNS do Docker para localização de serviço.



Passo 1: Criar os Recursos de Infraestrutura (Rede e Volume)



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Bash



```
# 1. Criar a rede isolada customizada
docker network create rede-integrada

# 2. Criar o volume gerenciado para os dados do banco
docker volume create pgdata-volume
```

Passo 2: Subir o Container do Banco de Dados PostgreSQL

Execute a imagem do PostgreSQL conectando-a à rede rede-integrada e montando o volume pgdata-volume:

Bash



```
docker run -d \
  --name db-postgres \
  --network rede-integrada \
  --mount source=pgdata-volume,target=/var/lib/postgresql/data \
  -e POSTGRES_USER=usuario_app \
  -e POSTGRES_PASSWORD=senha_extremamente_segura \
  -e POSTGRES_DB=meubanco \
  postgres:15-alpine
```

Validação: Verifique se o container do banco está em execução ativa:

Bash



```
docker ps
```



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Passo 3: Criar o Código da API Web Conectada ao Banco

Em uma nova pasta de projeto local, crie a estrutura da API:

1. Arquivo package.json:

JSON

```
{  
  "name": "api-com-banco",  
  "version": "1.0.0",  
  "main": "app.js",  
  "dependencies": {  
    "express": "^4.18.2",  
    "pg": "^8.11.3"  
  }  
}
```

Arquivo app.js:



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

JavaScript

```
const express = require('express');
const { Pool } = require('pg');

const app = express();
app.use(express.json());

// Configuração de conexão utilizando variáveis de ambiente
// Repare que o HOST padrão é o NOME do container do banco ('db-postgres')
const pool = new Pool({
  host: process.env.DB_HOST || 'db-postgres',
  port: process.env.DB_PORT || 5432,
  user: process.env.DB_USER || 'usuario_app',
  password: process.env.DB_PASSWORD || 'senha_extremamente_segura',
  database: process.env.DB_NAME || 'meubanco',
});

// Inicialização: Cria a tabela automaticamente se não existir
async function initDb() {
  try {
    await pool.query(`
      CREATE TABLE IF NOT EXISTS visitas (
        id SERIAL PRIMARY KEY,
        data TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
    `);
    console.log('Tabela no PostgreSQL verificada/criada com sucesso!');
  } catch (err) {
    console.error('Erro ao conectar ou inicializar o Banco de Dados:', err);
  }
}

// Rota 1: Registra uma nova visita no banco
app.post('/visita', async (req, res) => {
  try {
    const result = await pool.query('INSERT INTO visitas DEFAULT VALUES RETURNING *');
    res.status(201).json({ mensagem: 'Visita registrada!', registro: result.rows[0] });
  } catch (err) {
    res.status(500).json({ erro: err.message });
  }
});

// Rota 2: Lista todas as visitas salvas
app.get('/visitas', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM visitas');
    res.json({ total: result.rowCount, visitas: result.rows });
  } catch (err) {
    res.status(500).json({ erro: err.message });
  }
});

app.listen(3000, () => {
  console.log('API executando na porta 3000');
  initDb();
});
```



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Arquivo Dockerfile:

Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Passo 4: Compilar a Imagem da API

Compilar a imagem da API com a tag `api-banco:v1`:

Bash

```
docker build -t api-banco:v1 .
```

Passo 5: Executar o Container da API Conectado à Mesma Rede

Atente para a sinalização `--network rede-integrada`, que insere a API na mesma sub-rede virtual do banco de dados:



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Bash

```
docker run -d \  
  --name api-web \  
  --network rede-integrada \  
  -p 8080:3000 \  
  -e DB_HOST=db-postgres \  
  -e DB_USER=usuario_app \  
  -e DB_PASSWORD=senha_extremamente_segura \  
  -e DB_NAME=meubanco \  
  api-banco:v1
```

Passo 6: Testar a Integração da Aplicação Multicontainer

1. Enviar requisições POST para registrar visitas no banco:

Bash

```
curl -X POST http://localhost:8080/visita  
curl -X POST http://localhost:8080/visita
```

Resposta Esperada:

JSON

```
{ "mensagem": "Visita registrada!", "registro": { "id": 1, "data": "2026-10-02T13:30:00.000Z" } }
```

Consultar todas as visitas salvas via GET:



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Bash

```
curl http://localhost:8080/visitas
```



Passo 7: Prova da Persistência de Dados (Teste de Resiliência)

Para provar que a arquitetura está correta e que os dados do banco não dependem do ciclo de vida do container:

1. Destrua o container do Banco de Dados PostgreSQL:

Bash

```
docker stop db-postgres  
docker rm db-postgres
```



Crie um NOVO container de banco apontando para o MESMO volume (**pgdata-volume**):

Bash

```
docker run -d \  
  --name db-postgres \  
  --network rede-integrada \  
  --mount source=pgdata-volume,target=/var/lib/postgresql/data \  
  -e POSTGRES_USER=usuario_app \  
  -e POSTGRES_PASSWORD=senha_extremamente_segura \  
  -e POSTGRES_DB=meubanco \  
  postgres:15-alpine
```



Execute novamente a consulta na API:



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

Bash



```
curl http://localhost:8080/visitas
```

Resultado do Teste: Todos os registros gravados anteriormente permanecem **íntegros e acessíveis**, comprovando o sucesso do desacoplamento da camada de persistência através de **Managed Volumes**!

RESUMO DE COMANDOS DO MÓDULO

Bash



```
# Compilação e Imagens
docker build -t nome-imagem:tag .
docker image ls
docker tag imagem:tag usuario/repositorio:tag
docker push usuario/repositorio:tag

# Gerenciamento de Redes Customizadas
docker network create nome-da-rede
docker network ls
docker network inspect nome-da-rede

# Gerenciamento de Volumes
docker volume create nome-do-volume
docker volume ls
docker volume inspect nome-do-volume

# Execução Multicontainer Interconectada
docker run -d --name container-db --network minha-rede --mount source=meu-vol,target=
docker run -d --name container-web --network minha-rede -p 8080:3000 api-imagem
```



CONSTRUÇÃO DE IMAGENS E APLICAÇÕES COM DOCKER

REFERÊNCIAS

Livros e Obras de Referência

- **COMMUNITY, Docker.** *Docker in Action*. 2. ed. Manning Publications, 2019.
- **FRAYSER, Sean.** *Docker Deep Dive: Zero to Hero*. Independently published, 2023.
- **GARTNER, P.; SHARMA, S.** *Container Security: Fundamental Technology Concepts that Protect Containerized Applications*. O'Reilly Media, 2020.
- **KANE, Sean P.; MATTHIAS, Karl.** *Docker: Up & Running: Shipping Reliable Containers in Production*. 3. ed. Sebastopol: O'Reilly Media, 2023.
- **MOUAT, Adrian.** *Using Docker: Developing and Deploying Software with Containers*. Sebastopol: O'Reilly Media, 2015.
- **PULTORAK, David.** *Docker and Kubernetes for Java Developers*. Packt Publishing, 2021.
- **TURNBULL, James.** *The Docker Book: Containerization is the new virtualization*. OpenSource Press, 2014.

Recursos Web e Documentação Oficial

- **DOCKER.** *Docker Documentation*. Disponível em: <https://docs.docker.com/>. Acesso em: 07 out. 2026.
- **DOCKER HUB.** *Official Repositories and Image Registry*. Disponível em: <https://hub.docker.com/>. Acesso em: 07 out. 2026.
- **OPEN CONTAINER INITIATIVE (OCI).** *OCI Image Specification & Runtime Specification*. Disponível em: <https://opencontainers.org/>. Acesso em: 07 out. 2026.
- **SPRING.** *Spring Boot Reference Documentation: Efficient Container Images*. Spring / Broadcom. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/html/container-images.html>. Acesso em: 07 out. 2026.
- **SWAGGER / SMARTBEAR.** *OpenAPI Specification*. Disponível em: <https://swagger.io/specification/>. Acesso em: 07 out. 2026.